



OPEN ACCESS

Vietnam Journal of Computer Science

Vol. 12, No. 4 (2025) 469–488

© The Author(s)

DOI: [10.1142/S219688882450026X](https://doi.org/10.1142/S219688882450026X)



World Scientific
www.worldscientific.com

YOSCA: Confidence Adjustment for Better Object Detection in Aerial Images

Loi Nguyen

*Faculty of Information Technology, University of Science,
Ho Chi Minh City, Vietnam, University of Information Technology,
Vietnam National University Ho Chi Minh City, Vietnam
loinh@uit.edu.vn*

Khang Nguyen

*University of Information Technology,
Vietnam National University Ho Chi Minh City, Vietnam
khangnttm@uit.edu.vn*

Accepted 22 December 2024

Published 14 February 2025

Detecting small objects is a significant challenge when evaluating images captured from UAVs. Although YOLO models have been successful in detecting conventional objects, but they still face challenges in detecting small objects in traffic monitoring scenarios. Applying the Slicing-Aided Hyper Inference (SAHI) framework to YOLO models can expand the pixel regions containing small objects, thereby improving the detection of objects with tiny and small sizes. However, this can also lead to the cropping of larger vehicles, which reduces accuracy. This paper proposes a method that combines the results of the YOLO model with an SAHI-enhanced YOLO version, after adjusting the confidence scores of bounding boxes using a Gaussian Mixture Model-based approach, to improve accuracy in detecting small vehicles without compromising the detection of larger objects. This combined approach enhances detection coverage by normalizing the model outputs based on synthesized data, including removing duplicates and score normalization, and then merging the two detection sets. We conduct a thorough evaluation of the proposed method with different versions of YOLO: YOLOv5, YOLOv6, YOLOv7 and YOLOv8. Experimental results on two aerial surveillance datasets, namely, Visdrone and DroneVehicle, demonstrate the increased accuracy of mAP across all tested models.

Keywords: Object detection; YOLO; SAHI; small objects; aerial images.

*Corresponding author.

This is an Open Access article published by World Scientific Publishing Company. It is distributed under the terms of the [Creative Commons Attribution 4.0 \(CC BY\) License](https://creativecommons.org/licenses/by/4.0/), which permits use, distribution and reproduction in any medium, provided the original work is properly cited.

1. Introduction

Vehicle detection in images plays a vital role in intelligent traffic management systems. However, vehicle detection in video frames captured by traditional surveillance cameras (fixed or moving vehicle cameras) is limited in the surveillance scene's field of view. Benefiting from the rapid development of the global drone industry, crewless aerial vehicles (UAVs) have been applied in many fields such as security, surveillance, search and rescue, and sports analysis. Unlike traditional surveillance cameras, UAVs with moving cameras have inherent advantages such as easy deployment, high mobility, large field of view, and uniform aspect ratio. However, the fact that drones fly at different altitudes also brings many challenges to existing detection and tracking technologies, such as high density, small objects, and moving cameras. VisDrone¹ and DroneVehicle² are two large-scale crewless aerial vehicles (UAV) image datasets. The bounding boxes of the vehicles are carefully annotated to suit the real-time vehicle detection problem.

YOLO³ is a series of one-stage object detection methods which widely employed in object detection tasks. Their capability to swiftly and accurately detect objects makes them particularly well-suited for real-time traffic monitoring systems. However, despite the advancements seen in the YOLO family, especially with YOLOv5,⁴ YOLOv6,⁵ YOLOv7,⁶ and YOLOv8,⁷ their efficacy in detecting small objects remains significantly limited due to the challenge of extracting features from such objects. Therefore, YOLO models need to be improved to detect small vehicles in aerial images.

Slicing-Aided Hyper Inference (SAHI)⁸ introduced a slice-based framework that partitions images into smaller patches, effectively enlarging the pixel areas containing small objects, thereby improving the detection accuracy. Applying the SAHI framework to YOLO models has shown significantly better results in small object detection compared to the original YOLO models. However, the performance on large objects decreased because these objects tend to be fragmented across different image regions.

We here propose a mechanism, so-called **YOLO-based Confidence Adjustment** (YOSCA) that combines the results of the original YOLO model with the improved YOLO model using SAHI. This hybrid approach leverages the strengths of both models: the original YOLO for large object detection and the YOLO with SAHI for small object detection. To be specific, we implement a fine-tuning process using a Gaussian Mixture Model⁹ (Eq. (1)) to normalizes the confidence scores from both models, ensuring they are within a compatible range. Our fusion method improves the detections recall by normalizing the detection of the synthesized data based model (including duplicates truncating and scores rescaling), then concatenating both of detections set.¹⁰ This combined approach aims to maximize overall detection performance across varying object sizes, making it a robust solution for diverse object detection tasks.

The principal contributions of our paper are as follows:

- We propose YOSCA, a novel framework designed to detect objects captured from an aerial perspective. Our proposed framework is a combination of two results including one from YOLO detection result and one from SAHI framework applied to the YOLO models.
- Furthermore, we provide a comprehensive comparison and analysis of our method with respect to YOLO series models along with SAHI framework, which are applied, respectively, to those YOLO model on the VisDrone dataset and the DroneVehicle dataset.

2. Related Work

Detecting small and high-density objects in drone-captured video poses significant challenges. Recently, the SAHI framework has emerged as a promising solution to this problem. In the paper,⁸ the authors experimented with models such as VFNet,¹¹ TOOD,¹² and FCOS,¹³ and demonstrated the effectiveness of using slicing-aided fine-tuning and slicing-aided hyper inference methods. They indicated that Slicing-Aided Hyper Inference improves the performance of small object detection, although it may reduce the reliability in detecting larger objects. Furthermore, tuning the slice size can also improve the detection effectiveness.

In a separate investigation,¹⁴ applying SAHI to the Faster R-CNN model¹⁵ improved small object detection capabilities. Additionally, a new labeling strategy utilizing a Gaussian receptive field¹⁶ has been integrated into the Faster R-CNN algorithm to improve the accuracy of label assignments. Experiments carried out on the VisDrone dataset showed a significant improvement in accuracy (24.6%) and recall (38.3%) for small objects in the validation set. A different approach¹⁷ addresses the challenge of detecting small targets in satellite images (DOTA,¹⁸ TGRS-HRRSD,¹⁹ and RSOD²⁰ dataset). This method integrates Efficient Channel Attention (ECA)²¹ and Adaptively Spatial Feature Fusion (ASFF)²² into YOLOX to improve accuracy in the detection of small objects. The Varifocal Loss (VFL) function¹¹ is also used to mitigate the imbalance between positive and negative samples during training. Finally, this model is combined with the SAHI framework for large-image detection.

The combination strategy is an effective solution to overcome the limitations of both models. A notable example is the YADA method,²³ which uses one CNN model to train on “easy” objects and another CNN model to train on “hard” objects. The formula (1) is then applied to adjust the confidence scores from the detections. In the paper,²⁴ a method is proposed that merges high-scoring bounding boxes for object detection using a Gaussian Mixture Models⁹ and Non-Maximum Suppression (NMS).²⁵ Experimental results on 160 images with varying face and pedestrian sizes show that the IoU of the final detections increased by an average of 7.8% compared to NMS.

2.1. YOLO family

The advent of You Only Look Once (YOLO)³ marks a paradigm shift and has become a new trend in recent years. Unlike previous object detection methods, which used classifiers for detection, YOLO utilized an end-to-end neural network for simultaneous classification and predicted bounding-box. This innovative approach has helped YOLO achieve state-of-the-art results in real-time object detection.

Among the YOLO series, **YOLOv5**⁴ stands out as the most widely adopted version. Based on the advantages of the previous version^{3,27–29} in terms of detection speed and accuracy, YOLOv5 incorporates the “dynamic anchor boxes” method⁴ to refine anchor box alignment to the detected object shape and size. The EfficiencyDet architecture³⁰ increased accuracy and generalization in a variety of subjects. In addition, YOLOv5 also has a network structure comprising CSPDarkNet53,²⁹ SPPF,⁴ path aggregation network (PANet),³¹ ResNet³² blocks, etc.

YOLOv6⁵ is a single-stage object detection framework tailored for industrial applications. Its EfficiencyNet-V2 network architecture³³ has fewer parameters and higher efficiency than EfficiencyDet in YOLOv5, delivering superior detection accuracy and inference speed. By using an anchor-free method,¹³ YOLOv6 enhanced generalization and reduced post-processing overhead.

YOLOv7⁶ optimized both architecture and training process, combining three key features: E-ELAN for efficient training, model scaling to adapt to different sizes, and a “free gift bag” approach to balance precision and efficiency. E-ELAN controlled gradient paths for deep learning models, while model scaling adjusted hardware utilization properties.

YOLOv8²⁶ has several improvements introduced by Ultralytics in 2023. Through the use of YOLOv5’s backbone CSP, YOLOv8 replaced the C3 module with a C2f module.³⁴ For the box bounds classification and recovery tasks, YOLOv8 applied the Free Anchor technique with subsequent Task Alignment Learning. Furthermore, YOLOv8 removed branch objects from the network and used header splits for classification and classification. The VFL loss function¹¹ and the DFL loss function³⁵ were applied to the task and response types, respectively.

2.2. Slicing-Aided Hyper Inference (SAHI)

SAHI⁸ introduced a framework designed to improve the detection capabilities of small objects in models, as shown in Fig. 1. This framework consists of two main phases: slicing-aided fine-tuning (SF) and slicing-aided hyper inference, each of which plays a key role in improving detection accuracy.

The first phase, slicing-aided fine-tuning (SF), began by dividing the images in the fine-tuning dataset into smaller, manageable patches, denoted $I_1^F, I_2^F, \dots, I_j^F$. These patches were then further subdivided into overlapping sub-patches $P_1^F, P_2^F, \dots, P_k^F$. Each sub-patch was resized to larger dimensions, serving multiple purposes. First, it acted as a form of data augmentation, increasing the variety of training samples, and helping the model learn more robust features. Second, enlarging these patches

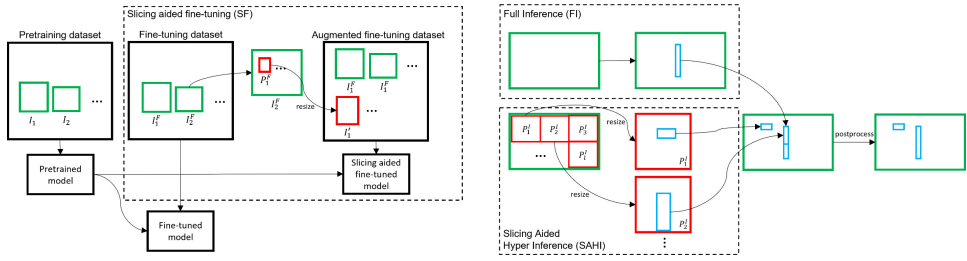


Fig. 1. Slicing-aided fine-tuning and slicing-aided hyper inference methods.⁸

amplified the features of small objects, making them more discernible during the training process. This meticulous slicing and resizing process was fundamental in enhancing the model's ability to detect small objects, which were often missed by conventional detection methods.

The second phase, slicing-aided hyper inference, involved a method of merging the overlapping sub-patches P^F back into their original image size. This was achieved through the Non-Maximum Suppression (NMS) function, a critical component that helps consolidate detection results by eliminating redundant detections and retaining the most relevant ones. The NMS function ensured that the merging process was seamless and that the final detection results were accurate and comprehensive. By incorporating the original images I^F along with the sliced patches, the framework ensured that the detection of large objects was not affected, effectively balancing the needs to detect both small and large objects within the same image.

3. Proposed Method

The YOLO model's result set often detects omissions or exhibits low reliability, particularly for small objects, resulting in low recall for these objects. Although the SAHI framework has addressed this weakness of the YOLO model, it reduces the reliability for large objects because they can be fragmented into multiple regions. Combining both result sets is necessary to increase recall. However, readjusting reliability is required due to the excessive number of overlapping predictions. The NMS function is a commonly used solution, but is ineffective in this case. In practice, this method only selects detections with high local scores, which sometimes leads to less accurate results. In this paper, we propose a new method (see Fig. 3) to adjust distributions using Gaussian Mixture Model (GMM) is a powerful method for modeling and refining data through distributions composed of multiple Gaussian (normal) distributions. GMM can effectively reduce noise and highlight important patterns or clusters in the data that a machine learning model aims to detect, thereby improving the model's recognition accuracy. We utilize Eq. (1) based on the GMM to adjust the confidence scores of bounding boxes across two result sets, subsequently merging them into a single unified set. This formula takes into account both spatial

information and the confidence score of each detection. The NMS function is applied as a post-processing step to evaluate the merged set using the pycocotools library. NMS eliminates redundant bounding boxes, retaining only the most relevant bounding box for each object. To address this issue, we first apply a confidence threshold to filter out and remove bounding boxes with low confidence scores and named it denoted $\mathcal{X}$. This step ensures that only the most reliable detections are considered for further processing.

Next, we readjust the confidence values of the bounding boxes for each object class, and we call this set $\mathcal{S}$. This consists of detections made by the YOLO + SAHI. The confidence re-adjustment is performed using the following formula:

$$\tilde{\mathcal{S}} = \mathcal{S} \times \frac{1}{\mu'_C} \times (1 - \mu_C - \gamma \times \sigma_C), \quad (1)$$

where

- μ_C is the average confidence of class $C \in \mathcal{X}$.
- μ'_C is the average confidence of bounding boxes for class $C \in \mathcal{S}$.
- γ is an adjustment coefficient helps in combining the two distributions of confidence values from $\mathcal{X}$ and $\mathcal{S}$.
- σ_C is the standard deviation of the confidence values for class C in the YOLO model's result set.

The adjustment formula is designed to normalize the confidence values in such a way that the combined result set reflects a more accurate and reliable detection performance. The parameter γ is particularly important as it balances the confidence distributions, ensuring that the strengths of both models are used effectively. In the experiments, we establish the value $\gamma = 0.2$.

To ensure the reliability of our combined detections, we impose the constraint $\mu_C > 0.5$. This constraint guarantees that the average confidence level of the YOLO model's detections is sufficiently high before they are merged with the YOLO + SAHI detections. Finally, we concatenate the filtered set $\mathcal{X}$ and the re-adjusted sets $\mathcal{S}$ to form the final detection input. This combined set is then evaluated using Pycocotools, which provides a comprehensive evaluation of the detection performance across various metrics. As a result, our method not only improves the detection of small objects but also maintains a high level of accuracy for larger objects. By integrating the YOLO result set and YOLO + SAHI results set in this manner, we leverage the strengths of both models to achieve a more robust and reliable object detection system, capable of handling a wide range of object sizes with improved accuracy. Adjusting confidence scores not only enhances detection performance for small objects but also improves results for medium and large objects. This is because the outputs of the two individual models complement each other's shortcomings or errors in detection, resulting in a unified result set with higher accuracy.

4. Datasets

VisDrone2019 dataset¹ is a large-scale dataset that comprises carefully annotated aerial images from UAVs and ground truth for various computer vision tasks. To collect data from locations with high object density, UAVs fly over approximately 14 major cities in China, including urban areas, arterial streets, highways, and intersections. This dataset consists of 8629 images in .jpeg format, which belong to 10 object classes: pedestrian, people, bicycle, car, van, truck, tricycle, awning tricycle, bus, and motor, with a total of 457,066 labels. According to MS COCO's dataset standards, we divide the image set into three folders: train (6471 images), test (1610 images), and validation (548 images), along with the label set, divided correspondingly into train (343,205 labels), validation (38,759 labels) and testing (75,102 labels) folders.

DroneVehicle dataset² is also a large-scale captured by drone perspective. This dataset consists of 56,878 images, divided into two halves: RGB images and infrared images. We use RGB images similar to the VisDrone dataset for the experiments. The dataset has 28,439 labels and annotated images for 5 object classes: car (389,779 annotations), freight car (13,400 annotations), truck (22,123 annotations), bus (15,333 annotations), van (1935 annotations). According to MS COCO data standards, we divide images are placed into three folders: train (17,990 images), test (8980 images) and validation (1469 images), along with a set of labels, divided into train folder (17,990 labels), validation folder (8980 labels) and test folder (1469 labels), respectively.

5. Experiments and Results

Based on the length of the residual structure, the YOLO model is categorized into five sizes: n, s, m, l, and x. In Fig. 2, The x-size yields the highest accuracy, although the YOLOv6 version does not include an x-size variant. To improve accuracy for small objects detection, we use the most advanced models at an image size of 640×640 , such as YOLOv5x, YOLOv6l, YOLOv7x, and YOLOv8x, and then fine-tuning each of the models to increase their precision.

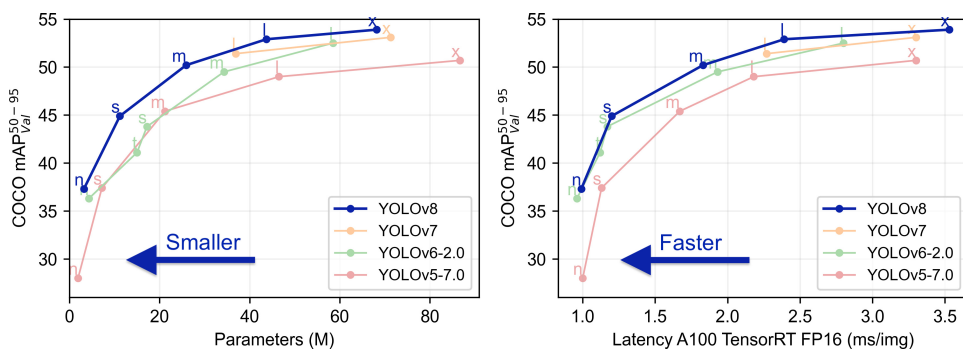


Fig. 2. Compare YOLO models.²⁶

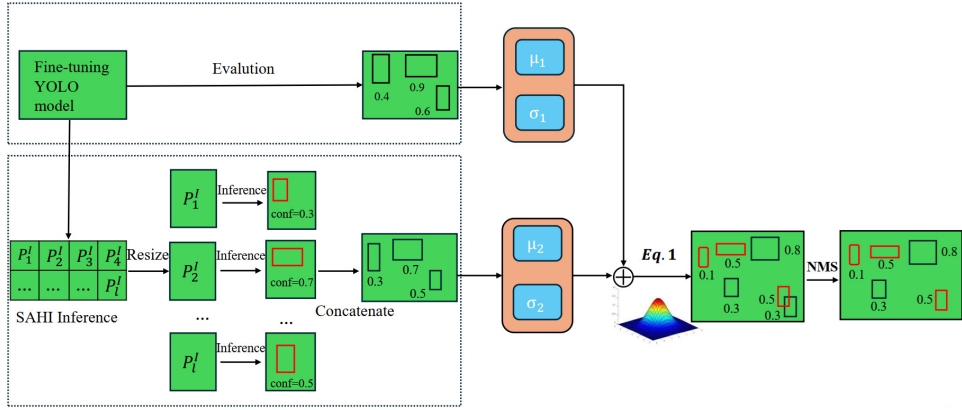


Fig. 3. Architecture of our proposed method.

5.1. Experimental settings

In this test, the input image size is set to 640×640 and is performed on a server running the Ubuntu 20.04.1 LTS operating system with a 2 GeForce GTX 2080ti 11GB GPU. The results are recorded after 300 epochs, with a batch size of 4 for each epoch. The hyperparameters in the YOLO model and cost functions remained consistent throughout the experiment.

5.2. Evaluation metrics

The mean Average Precision (mAP) according to the MS COCO standard serves as the evaluation metric in our paper. We utilize two measurements, mAP@0.5 and mAP@0.5:0.95, to objectively compare models. Here, mAP@0.5 represents the average precision when the Intersection over Union (IOU) is 0.5, while mAP@0.5:0.95 represents the mean value of mAP across IOU thresholds ranging from 0.5 to 0.95 with a step size of 0.05. To calculate each mAP measure as described above, the following procedure is followed:

$$P = TP / (TP + FP), \quad (2)$$

$$AP = \int_0^1 P(R), \quad (3)$$

$$mAP = \frac{1}{c} \sum_{j=1}^c AP_i, \quad (4)$$

$$mAP@0.5 = \frac{\sum_{j=1}^c AP@0.5_j}{c} \quad (5)$$

$$mAP@0.5 : 0.95 = \frac{mAP@0.5 + mAP@0.55 + \dots + mAP@0.95}{10}, \quad (6)$$

where

- P is Precision.
- TP is True positive.
- FP is False positive.
- FN is False negative.
- c is Numbers of classes.
- AP_j is Average accuracy of class j .
- mAP is Average mAP value of the average precision.

5.3. Experimental results

In our implementation of the SAHI framework on fine-tuned YOLO models using the VisDrone dataset, we evaluated slice sizes of 256×256 , 512×512 , and 640×640 . Our findings indicate that the 512×512 slice size yielded superior results compared to the other two sizes (as illustrated in Figs. 4–7). Subsequently, we conduct further experiments with the SAHI framework on YOLO models, maintaining a slice size of 512×512 and an overlap ratio (width $\times$ height) of 0.2×0.2 , while varying

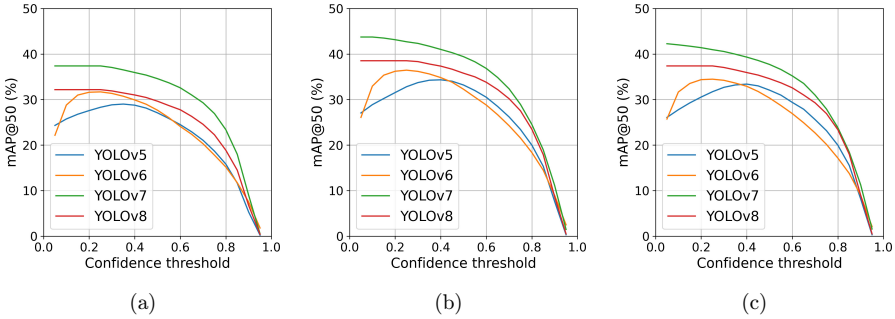


Fig. 4. Accuracy mAP@50 of SAHI framework on YOLO models are fine-tuned on the VisDrone dataset. Slice sizes 256×256 (a), 512×512 (b), and 640×640 (c) (ratio 0.2×0.2 overlap).

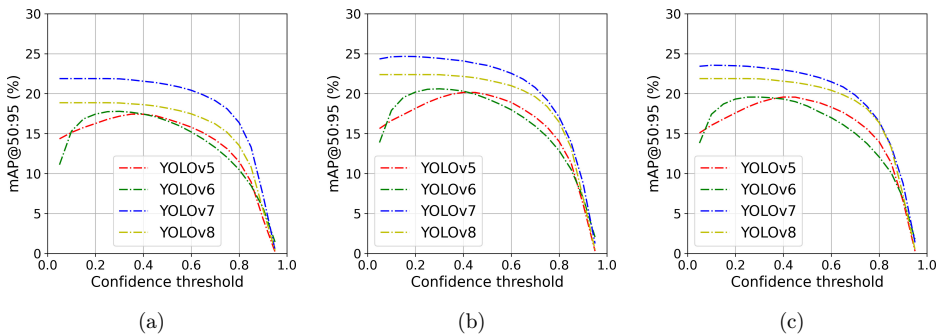


Fig. 5. Accuracy mAP@50:95 of SAHI framework on YOLO models are fine-tuned on the VisDrone dataset. Slice sizes 256×256 (a), 512×512 (b), and 640×640 (c) (ratio 0.2×0.2 overlapping).

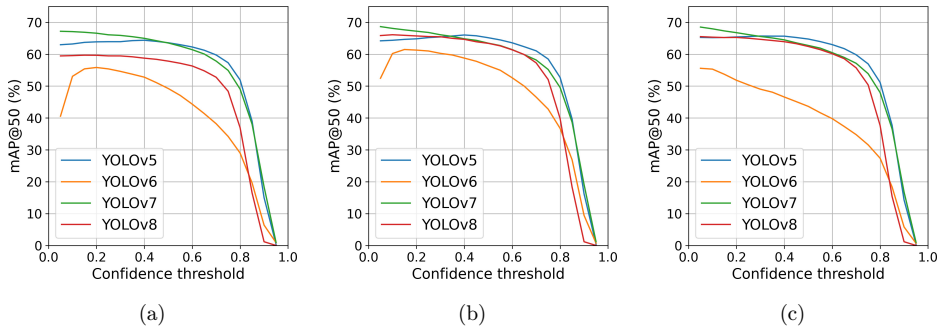


Fig. 6. Accuracy mAP@50 of SAHI framework on YOLO models are fine-tuned on the DroneVehicle dataset. Slice sizes 256×256 (a), 512×512 (b), and 640×640 (c) (ratio 0.2×0.2 overlapping).

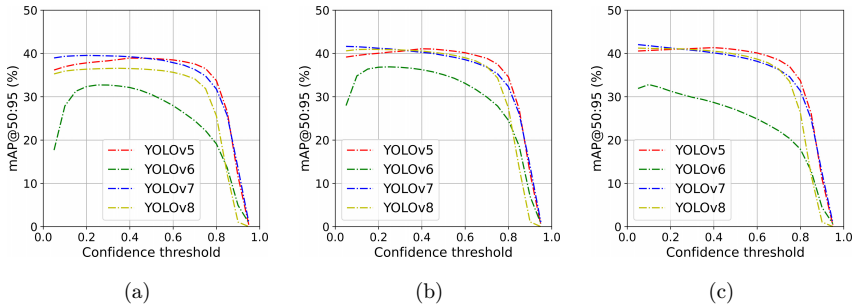


Fig. 7. Accuracy mAP@50:95 of SAHI framework on YOLO models are fine-tuned on the DroneVehicle dataset. Slice sizes 256×256 (a), 512×512 (b), and 640×640 (c) (ratio 0.2×0.2 overlapping).

confidence thresholds from 0.05 to 0.95 with increments of 0.05. Applying the SAHI framework improved accuracy across all YOLO variants such as YOLOv5, YOLOv6, YOLOv7 and YOLOv8). We observed peak performance with mAP_{50}^{test} values of 34.33%, 36.4%, 43.73%, and 38.52%, respectively, and $mAP_{50:95}^{test}$ values of 20.14%, 20.54%, 24.61%, and 22.38%, respectively. Specifically, YOLOv5 + SAHI achieved optimal accuracy at a confidence threshold of 0.45, while YOLOv6 + SAHI, YOLOv7 + SAHI, and YOLOv8 + SAHI demonstrated peak performance at confidence thresholds of 0.25, 0.1, and 0.25, respectively. We replicated this approach on the DroneVehicle dataset and found a gradual decrease in accuracy across all YOLO models when using the SAHI framework. Optimal results were consistently achieved with a slice size of 512×512 and an overlap ratio of 0.2×0.2 . The most effective confidence thresholds were 0.45 for YOLOv5 + SAHI, and 0.15, 0.05, and 0.1 for YOLOv6 + SAHI, YOLOv7 + SAHI, and YOLOv8 + SAHI, respectively. We attribute this performance variation to the dataset's composition, which exclusively contains large, real-sized vehicle objects across its five classes. The bounding boxes from the prediction results of the fine-tuned YOLO models were filtered using appropriate confidence thresholds ranging from 0.05 to 0.95, with 0.05 increments.

These filtered results were then utilized to recalibrate the confidence scores for the bounding boxes in the SAHI framework's prediction set across all fine-tuned YOLO models (YOLOv5, YOLOv6, YOLOv7 and YOLOv8) using Eq. (1). Finally, we aggregated the bounding boxes from both result sets to perform our evaluation.

Table 1 uses mAP metric to show a performance comparison of various methods, including the YOLOv5, YOLOv6, YOLOv7, YOLOv8, the YOLO versions from YOLOv5 to YOLOv8 + SAHI, and ours proposed models for object detection task on the VisDrone and the DroneVehicle datasets. In general, of the twelve well-known methods, the mAP of the YOSCAv7 model based on the YOLOv7 and SAHI framework is the highest in two benchmark datasets. As indicated clearly from the VisDrone dataset, the YOSCAv7 model surpasses the other state-of-the-art methods on AP_Small, AP_Medium, AP_Large, AP_50^{test}, and AP_50:95^{test}, ranging from 2.00% to 5.50%, whereas the YOLOv5 model is the lowest figure on the same metric in all classes at over 9.50%, at nearly 30.00%, 42.50%, roundly 34.00%, and at approximately 19.50%, respectively. Furthermore, the mAP of the YOLOv5 method is slightly lower than YOSCAv5, ranging from 0.2% to 0.4%, respectively. Remarkably, YOSCAv6 and YOSCAv8 methods are the same figures and the percentage of these models are slightly higher than that of the YOLOv6 and the YOLOv8 models, about 0.2% to 0.5%, respectively. Moreover, the number of methods proposed by us is also higher than that of YOLOv5 + SAHI, YOLOv6 + SAHI, YOLOv7 + SAHI, and YOLOv8 + SAHI in all classes, ranging from about 2.00% to 4.00%, respectively. Regarding the DroneVehicle dataset, it is clear that the highest percentage of the YOSCAv7 method accounts for 73.71% and more than 48.61%, and these figures exceed the other methods, from 3.00% to 4.00% on AP_50test, and AP_50:95test, respectively. Notably, the YOLOv6 model has the lowest result on AP_Small, AP_Medium, AP_Large, AP_50test, and AP_50:95test, at 27.50%, at 44.60%, at 25.10%, at just over 65.08%, at over 42.73%, respectively. Furthermore, the percentage of the YOSCAv8 method is higher than that of YOSCAv6 method at about 4.00% on AP_50test, and AP_50:95test. In addition, the figure of the YOLOv5 method is lowest in all classes. From the results presented in this table, it can be seen that our proposed methods obtain the best object detection performance for small objects, medium objects, and large objects compared to existing methods on two benchmark datasets. A detailed analysis of the figures shows that our proposed methods significantly outperform existing methods, which are based on YOLO versions and YOLO versions + SAHI for object detection tasks in drone-based video surveillance systems.

It may be observed from Table 2 that our proposed method based on combining the results of YOLO model and SAHI framework takes more time (ms) to process video frames than other methods on two benchmark datasets. As indicated clearly from the VisDrone dataset, YOLO methods namely YOLOv5 YOLOv6, YOLOv7 and YOLOv8 have the lowest inference time for each frame at just over 19.0 ms, 7.66 ms, 5.7 ms, and 19.8 ms, respectively. Similarly, these prominent methods also have the lowest processing time for each image on the DroneVehicle dataset at just

Table 1. Experimental results in terms of AP metric of YOLO, SAHI applied YOLO and ours proposed YOSCA. The best results are highlighted in bold.

Methods	VisDrone ¹					DroneVehicle ²				
	AP_Small	AP_Medium	AP_Large	AP_50 ^{test}	AP_50:95 ^{test}	AP_Small	AP_Medium	AP_Large	AP_50 ^{test}	AP_50:95 ^{test}
YOLOv5 ⁴	9.60	29.80	42.50	33.94	19.36	30.70	48.30	29.30	68.83	45.95
YOLOv5 + SAHI	12.00	28.80	35.90	34.02	20.14	25.10	45.10	20.70	65.78	41.01
YOSCAv5	9.90	30.10	42.60	34.35	19.66	32.00	50.90	29.90	72.88	48.27
YOLOv6 ⁵	10.30	34.70	54.80	36.51	22.16	27.50	44.60	25.10	65.08	42.73
YOLOv6 + SAHI	12.10	30.30	44.50	36.45	20.54	23.10	39.70	12.40	61.48	36.38
YOSCAv6	10.60	34.90	54.80	37.07	22.49	28.48	46.80	25.30	68.35	44.62
YOLOv7 ⁶	13.10	37.20	51.30	42.40	24.81	31.30	48.30	31.00	69.08	45.97
YOLOv7 + SAHI	15.50	34.80	45.00	43.73	24.61	26.80	45.50	22.50	68.66	41.62
YOSCAv7	13.40	37.50	51.50	42.85	25.15	32.90	51.20	31.70	73.71	48.61
YOLOv8 ⁷	10.70	33.90	52.30	37.41	21.98	31.20	48.40	30.90	68.49	46.26
YOLOv8 + SAHI	13.80	32.10	39.40	38.52	22.38	27.00	45.20	19.00	66.11	40.88
YOSCAv8	11.20	34.50	52.40	38.23	22.51	32.70	51.30	31.50	72.98	48.88

Table 2. Compare inference time of the prominent methods and our proposed method.

Dataset	Method	Inference time (ms)
VisDrone ¹	YOLOv5	19.1
	YOLOv5 + SAHI	370.0
	YOSCAv5	789.1
	YOLOv6	7.6
	YOLOv6 + SAHI	310.0
	YOSCAv6	867.7
	YOLOv7	5.7
	YOLOv7 + SAHI	260.0
	YOSCAv7	775.7
	YOLOv8	19.8
DroneVehicle ²	YOLOv8 + SAHI	390.0
	YOSCAv8	809.8
	YOLOv5	16.7
	YOLOv5 + SAHI	100.0
	YOSCAv5	676.7
	YOLOv6	11.15
	YOLOv6 + SAHI	90.0
	YOSCAv6	1085.4
	YOLOv7	8.3
	YOLOv7 + SAHI	70.0
	YOSCAv7	539.1
	YOLOv8	19.2
	YOLOv8 + SAHI	100.0
	YOSCAv8	520.8

over 16.7 ms, 11.15 ms, 8.3 ms, and about 19.0 ms, respectively. On the other hand, our method has the highest inference time for each video frame on these datasets and is much higher than the other well-known methods. Remarkably, YOSCAv6 method has highest inference time per frame compared to other methods on the two datasets at over 867.0 ms for video frame on the VisDrone dataset, and roundly 1085 ms per frame on the DroneVehicle dataset.

Table 3. The accuracy and inference time of the YOLO alone methods with the 960 × 960 image size from the DroneVehicle dataset.

DroneVehicle ²							
Image size	Methods	AP_Small	AP_Medium	AP_Large	AP_50 ^{test}	AP_50:95 ^{test}	Inference time (ms)
960 × 960	YOLOv5 ⁴	30.50	47.30	22.10	68.40	44.90	29.90
	YOLOv6 ⁵	28.70	43.70	17.80	63.80	41.60	22.21
	YOLOv7 ⁶	31.20	47.90	25.70	68.40	45.50	18.60
	YOLOv8 ⁷	31.00	47.30	21.90	67.50	45.00	42.10
640 × 640	YOSCAv5	32.00	50.90	29.90	72.88	48.20	676.70
	YOSCAv6	28.48	46.80	25.30	68.35	44.62	1085.40
	YOSCAv7	32.90	51.20	31.70	73.71	48.61	539.10
	YOSCAv8	32.70	51.30	31.50	72.98	48.88	520.80

We have conducted the experiment and observed that when comparing YOSCA with an input image size of 640 and YOLO individually with an input image size of 960 on the DroneVehicle dataset (see Tables 1–3), YOSCA consistently outperforms YOLO across all metrics. Although increasing the input image size by 1.5 times nearly doubles the inference time, all YOLO versions experienced a drop in accuracy compared to YOLO with a 640 image size. YOLOv7 alone is the version with the highest accuracy compared to the others, while still maintaining the fastest execution speed.

6. Discussion

The object classes in the DroneVehicle dataset are predominantly large in real-world dimensions. We observed that most of the captured images contain multiple cars, which are well-detected by the standalone YOLO model. Conversely, slicing images with SAHI often causes large objects to be fragmented into multiple regions, leading to a loss of critical information at the boundaries of slices and making detection more challenging. When a large object spans the boundary of slices, the model may identify it as separate fragments, resulting in duplicate or overlapping bounding boxes. YOLO and similar detection models rely on contextual information around an object to accurately determine its bounding box. When images are sliced, the context for large objects may be lost, leading to incomplete or inaccurate detections. The proposed method filters out bounding boxes with low confidence scores from the results of the standalone YOLO model using an appropriate confidence threshold. These are then replaced with bounding boxes from SAHI's results that have higher confidence scores. Subsequently, the confidence scores of SAHI's bounding boxes are adjusted based on the standalone YOLO results. This process highlights bounding boxes for small objects while suppressing bounding boxes for large objects from SAHI during the post-processing step using Non-Maximum Suppression (NMS). With proper settings, experiments show that our proposed approach significantly outperforms the current state-of-the-art for object detection tasks from drone videography. YOLO methods tend to overlook many small or obscured objects, such as pedestrians, motorcycles, resulting in sparse bounding boxes as depicted in Figs. 8(b), 9(b), 10(b) and 11(b), respectively. In contrast, Figs. 8(c), 9(c), 10(c) and 11(c) demonstrate the effectiveness of the SAHI framework with the YOLO methods in detecting a significant portion of objects with dense bounding box coverage. Despite the effectiveness of the SAHI framework in many cases, a white car next to a blue bus remains undetected when applied to the YOLOv5 and YOLOv7 models. The YOLOv6 model incorrectly identified the white car as a truck, with the bounding box not matching the actual object, while only the YOLOv8 model combined with the SAHI framework correctly locates the car. It is evident from the experimental results on two challenging detection datasets that by combining the set of results of the YOLO model with that of the SAHI framework after adjusting the confidence threshold of the bounding boxes with the Gaussian mixture model

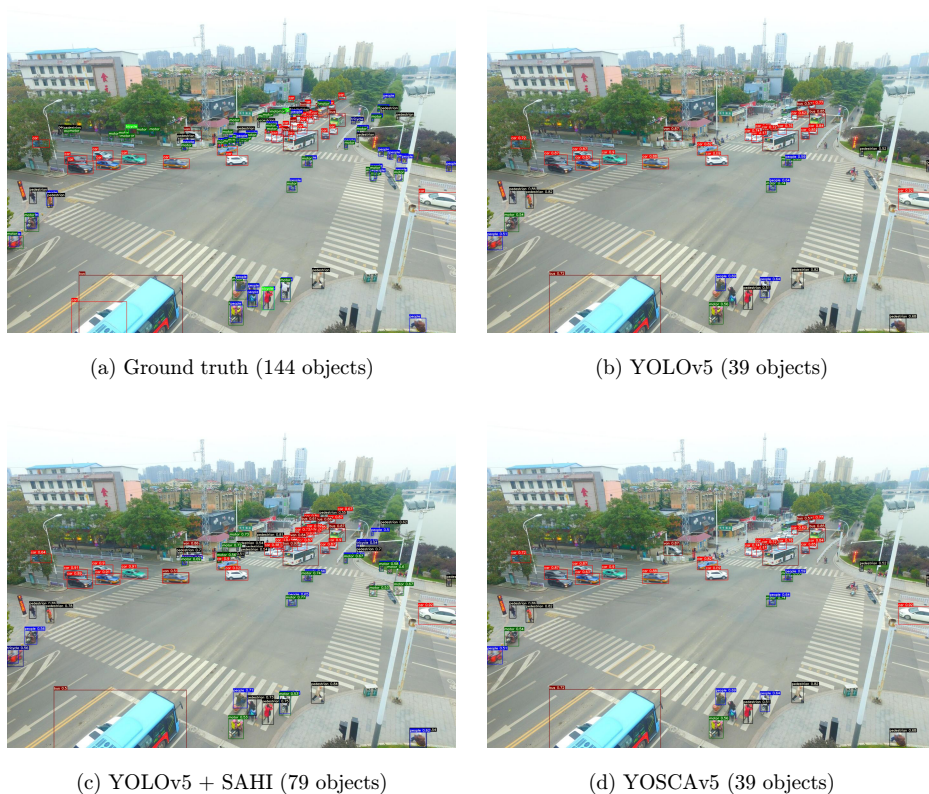


Fig. 8. Visualization of Ground truth (a), YOLOv5 (b), YOLOv5 + SAHI (c) and our combined method (d) (with confidence threshold 0.5).

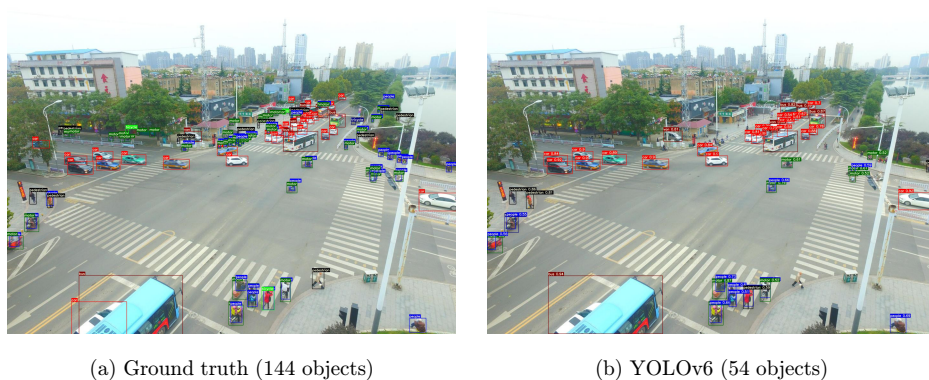


Fig. 9. Visualization of Ground truth (a), YOLOv6 (b), YOLOv6 + SAHI (c), and ours proposed YOSCAv6 (d) (with confidence threshold 0.5).



(c) YOLOv6 + SAHI (92 objects)



(d) YSeAv6 (54 objects)

Fig. 9. (Continued)



(a) Ground truth (144 objects)



(b) YOLOv7 (64 objects)



(c) YOLOv7 + SAHI (97 objects)



(d) YSeAv7 (64 objects)

Fig. 10. Visualization of Ground truth (a), YOLOv7 (b), YOLOv7 + SAHI (c), and ours proposed YSeAv7 (d) (with 0.5 confidence threshold).

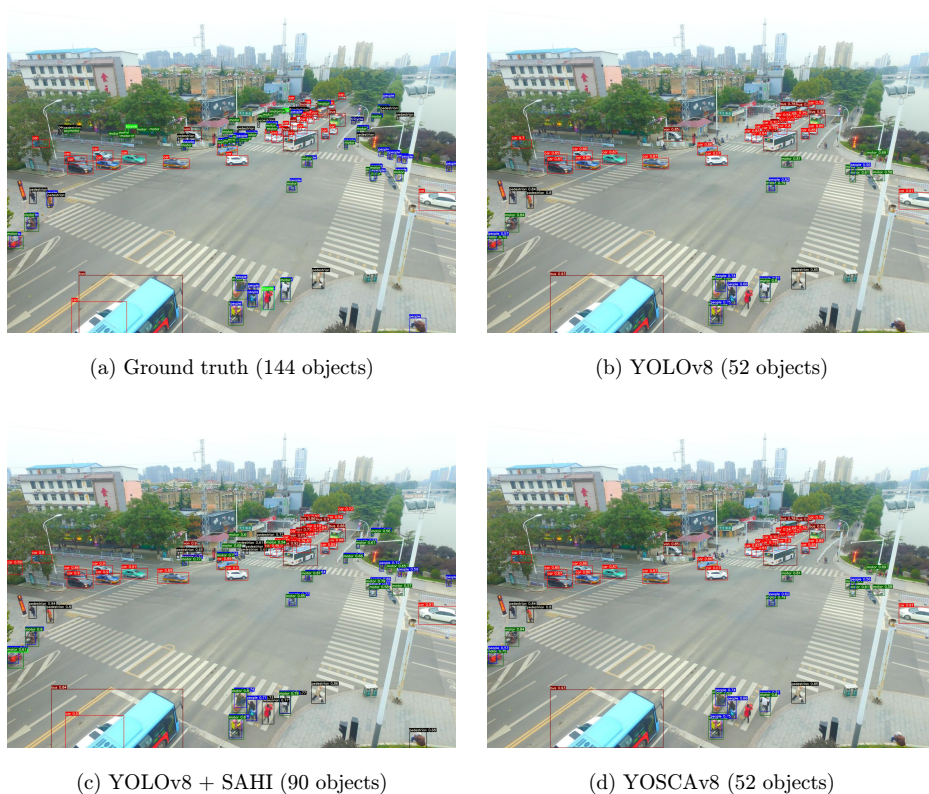


Fig. 11. Visualization of ground truth (a), YOLOv8 (b), YOLOv8 + SAHI (c) and our combined method (d) (with 0.5 confidence threshold).

(Eq. (1)), our method has fewer bounding boxes but detects a similar number of objects and achieves higher performance compared to other state-of-the-art object detection methods. Although the SAHI framework on the YOLO model tends to detect small objects with higher confidence scores than the YOLO model alone, our hybrid method preferentially retains the bounding boxes of the YOLO model. Furthermore, for larger objects, such as the blue car, the SAHI framework in the YOLO model generates detection results with lower confidence scores than in the YOLO model. In such cases, our approach prioritizes retaining the bounding boxes from the YOLO model, as shown in Figs. 8(d), 9(d), 10(d) and 11(d), respectively.

7. Conclusion and Future Work

In this paper, we introduce an approach for drone-captured object detection by combining the results of the YOLO models and the SAHI framework to select a more appropriate set of bounding boxes by applying a confidence threshold to filter and remove bounding boxes with low confidence scores. We then re-adjust the confidence

values of the bounding boxes for each object class, then re-adjust the sets to form the final detection result set. Therefore, our approach seeks to mitigate these issues by selecting bounding boxes more carefully from both models' results. In the future, other kinds of datasets in aerial videos and various experiments with different strategies will be conducted to validate the proposed approach further and promote in-depth research on object detection in practical aerial applications.

Acknowledgement

This research is funded by University of Information Technology - Vietnam National University Ho Chi Minh City under grant number D1-2024-71.

ORCID

Loi Nguyen  <https://orcid.org/0009-0005-7403-6887>

Khang Nguyen  <https://orcid.org/0000-0002-6571-7075>

References

1. D. Du *et al.*, Visdrone-det2019: The vision meets drone object detection in image challenge results, in *Proc. IEEE/CVF Int. Conf. Computer Vision Workshops* (IEEE, 2019), pp. 213–226.
2. Y. Sun, B. Cao, P. Zhu and Q. Hu, Drone-based rgb-infrared cross-modality vehicle detection via uncertainty-aware learning, *IEEE Trans. Circuits Syst. Video Technol.* **32**(10) (2022) 6700–6713.
3. J. Redmon, S. Divvala, R. Girshick and A. Farhadi, You only look once: Unified, real-time object detection, in *Proc. IEEE Conf. Computer Vision and Pattern Recognition* (IEEE, 2016) pp. 779–788.
4. G. Jocher *et al.*, ultralytics/yolov5: v7. 0-YOLOV5 SOTA realtime instance segmentation (Zenodo, 2022).
5. C. Li *et al.*, YOLOV6: A single-stage object detection framework for industrial applications, preprint (2022), arXiv:2209.02976.
6. C.-Y. Wang, A. Bochkovskiy and H.-Y. M. Liao, YOLOV7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors, in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition* (IEEE, 2023), pp. 7464–7475.
7. G. Jocher, A. Chaurasia and J. Qiu, Yolo by ultralytics (2023), <https://github.com/ultralytics/ultralytics>.
8. F. C. Akyon, S. O. Altinuc and A. Temizel, Slicing aided hyper inference and fine-tuning for small object detection, in *2022 IEEE Int. Conf. Image Processing (ICIP)* (IEEE, 2022), pp. 966–970.
9. D. A. Reynolds *et al.*, Gaussian mixture models, in *Encyclopedia of Biometrics*, Vol. **741** (Springer, 2009), pp. 659–663.
10. K.-D. Nguyen, K. Nguyen, D.-D. Le, D. A. Duong and T. V. Nguyen, You always look again: Learning to detect the unseen objects, *J. Vis. Commun. Image Represent.* **60** (2019) 206–216.
11. H. Zhang, Y. Wang, F. Dayoub and N. Sunderhauf, VariFocalNet: An IoU-aware dense object detector, in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition* (IEEE, 2021), pp. 8514–8523.

12. C. Feng, Y. Zhong, Y. Gao, M. R. Scott and W. Huang, TOOD: Task-aligned one-stage object detection, in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)* (IEEE Computer Society, 2021), pp. 3490–3499.
13. Z. Tian, C. Shen, H. Chen and T. He, FCOS: Fully convolutional one-stage object detection, in *Proc. IEEE/CVF Int. Conf. Computer Vision* (IEEE, 2019), pp. 9627–9636.
14. L. Chen, N. Cang, H. Jiang and S. Wang, Improved Faster R-CNN for dense small objects, in *Proc. 4th Int. Conf. Frontiers Technology of Information and Computer (ICFTIC)* (IEEE, 2022), pp. 1056–1060.
15. S. Ren, K. He, R. Girshick and J. Sun, Faster R-CNN: Towards real-time object detection with region proposal networks, *Adv. Neural Inf. Process. Syst.* **28** (2015) 1137–1149.
16. C. Xu, J. Wang, W. Yang, H. Yu, L. Yu and G.-S. Xia, RFLA: Gaussian receptive field based label assignment for tiny object detection, in *Proc. European Conf. Computer Vision* (Springer, 2022), pp. 526–543.
17. L. Yang, G. Yuan, H. Zhou, H. Liu, J. Chen and H. Wu, RS-YOLOX: A high-precision detector for object detection in satellite remote sensing images, *Appl. Sci.* **12**(17) (2022) 8707.
18. G.-S. Xia, X. Bai, J. Ding, Z. Zhu, S. Belongie, J. Luo, M. Datcu, M. Pelillo and L. Zhang, DOTA: A large-scale dataset for object detection in aerial images, in *Proc. IEEE Conf. Computer Vision and Pattern Recognition* (IEEE, 2018), pp. 3974–3983.
19. Y. Zhang, Y. Yuan, Y. Feng and X. Lu, Hierarchical and robust convolutional neural network for very high-resolution remote sensing object detection, *IEEE Trans. Geosci. Remote Sens.* **57**(8) (2019) 5535–5548.
20. Z. Xiao, Q. Liu, G. Tang and X. Zhai, Elliptic Fourier transformation-based histograms of oriented gradients for rotationally invariant object detection in remote-sensing images, *Int. J. Remote Sens.* **36**(2) (2015) 618–644.
21. Q. Wang, B. Wu, P. Zhu, P. Li, W. Zuo and Q. Hu, ECA-Net: Efficient channel attention for deep convolutional neural networks, in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition* (IEEE, 2020), pp. 11534–11542.
22. S. Liu, D. Huang and Y. Wang, Learning spatial fusion for single-shot object detection, preprint (2019), arXiv:1911.09516.
23. K.-D. Nguyen, K. Nguyen, D.-D. Le, D. A. Duong and T. V. Nguyen, YADA: You always dream again for better object detection, *Multimedia Tools Appl.* **78**(19) (2019) 28189–28208.
24. C. Gu, T. Nakane, T. Akashi and C. Zhang, Merging scored bounding boxes with gaussian mixture model for object detection, in *6th IIAE Int. Conf. Intelligent Systems and Image Processing (ICISIP)* (IIAE, 2018), pp. 249–255.
25. J. Hosang, R. Benenson and B. Schiele, Learning non-maximum suppression, in *Proc. IEEE Conf. Computer Vision and Pattern Recognition* (IEEE, 2017), pp. 4507–4515.
26. G. Jocher, A. Chaurasia and J. Qiu, Ultralytics YOLO (2023).
27. J. Redmon and A. Farhadi, YOLO9000: Better, faster, stronger, in *Proc. IEEE Conf. Computer Vision and Pattern Recognition* (IEEE, 2017), pp. 7263–7271.
28. J. Redmon and A. Farhadi, YOLOv3: An incremental improvement, preprint (2018), arXiv:1804.02767.
29. A. Bochkovskiy, C.-Y. Wang and H.-Y. M. Liao, YOLOv4: Optimal speed and accuracy of object detection, preprint (2020), arXiv:2004.10934.
30. M. Tan and Q. Le, EfficientNet: Rethinking model scaling for convolutional neural networks, in *Proc. Int. Conf. Machine Learning* (PMLR, 2019), pp. 6105–6114.
31. S. Liu, L. Qi, H. Qin, J. Shi and J. Jia, Path aggregation network for instance segmentation, in *Proc. IEEE Conf. Computer Vision and Pattern Recognition* (IEEE, 2018), pp. 8759–8768.
32. K. He, X. Zhang, S. Ren and J. Sun, Deep residual learning for image recognition, in *Proc. IEEE Conf. Computer Vision and Pattern Recognition* (IEEE, 2016), pp. 770–778.

33. M. Tan and Q. Le, EfficientNetV2: Smaller models and faster training, in *Proc. Int. Conf. Machine Learning* (PMLR, 2021), pp. 10096–10106.
34. B. Xiao, M. Nguyen and W. Q. Yan, Fruit ripeness identification using YOLOv8 model, *Multimedia Tools Appl.* **83**(9) (2024) 28039–28056.
35. X. Li, W. Wang, L. Wu, S. Chen, X. Hu, J. Li, J. Tang and J. Yang, Generalized focal loss: Learning qualified and distributed bounding boxes for dense object detection, *Adv. Neural Inf. Process. Syst.* **33** (2020) 21002–21012.